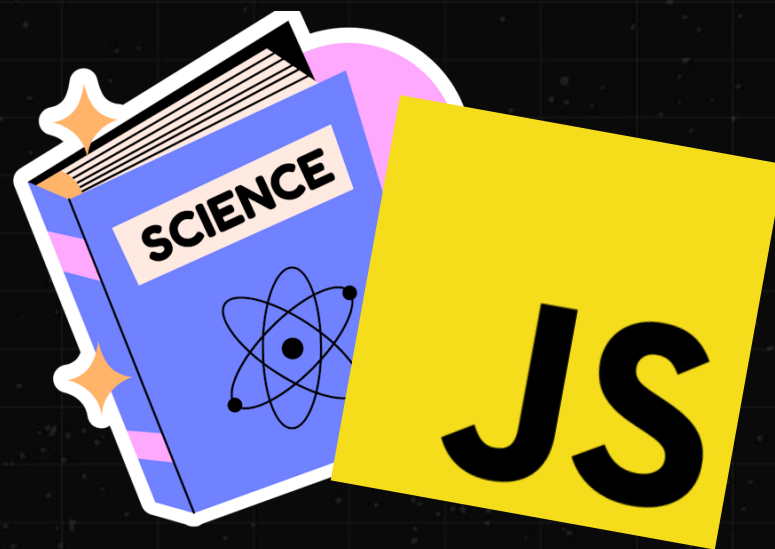


ЛЕКЦИЯ #3

CS BO FRONTEND

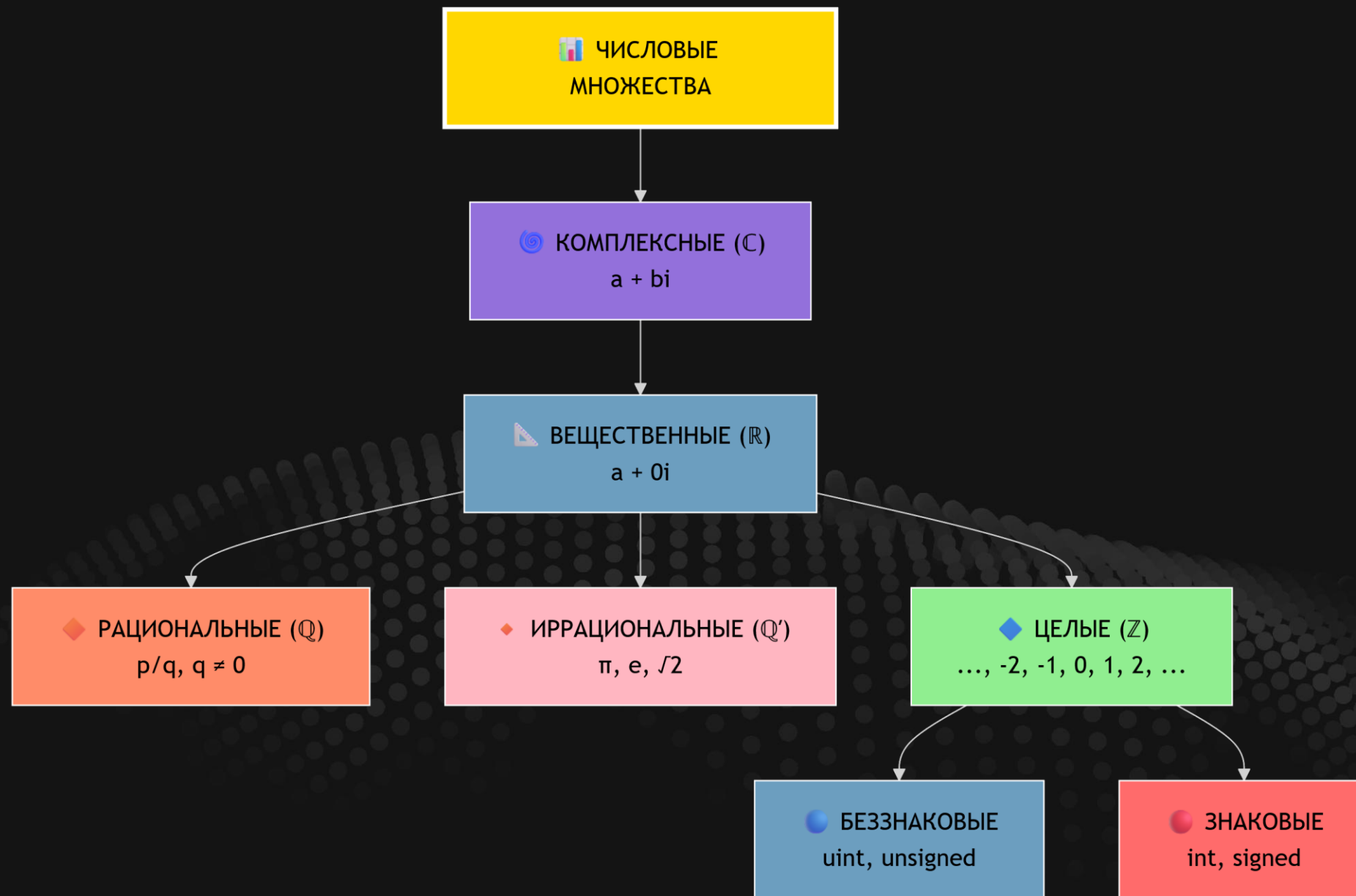


ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Поговорить про **виды числовых типов** и **способы их кодирования**
- * Разобрать **числовые типы** в JavaScript
- * Познакомиться с **типизированными массивами** в JavaScript

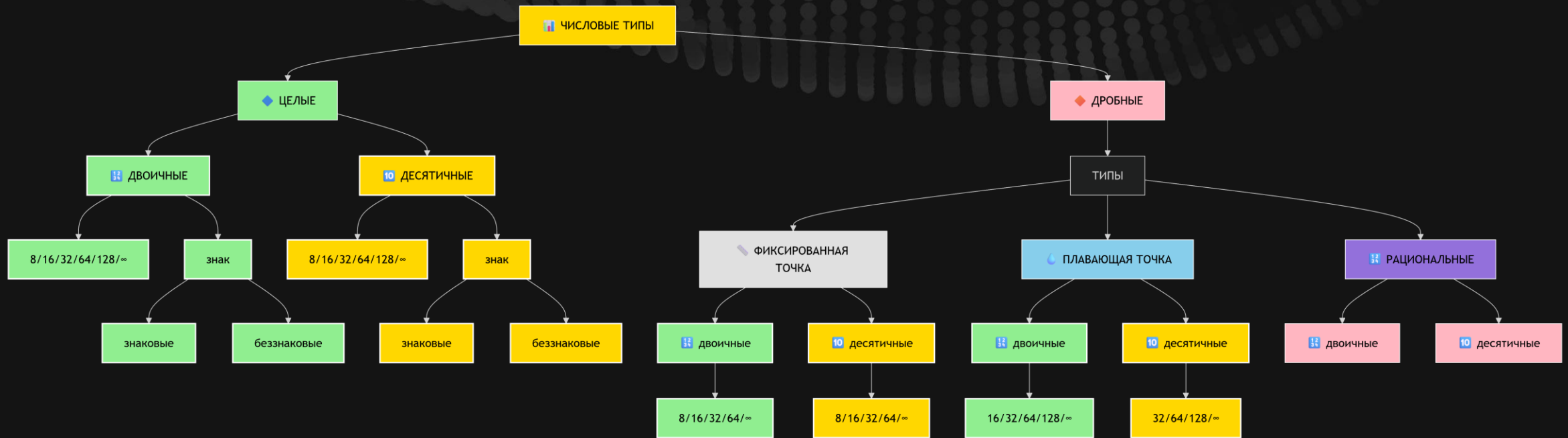
А КАКИЕ ЕСТЬ ЕЩЕ
ЧИСЛА **КРОМЕ**
ЦЕЛЫХ?





РЕФЛЕКСИРУЕМ

- * С математической точки зрения существуют **большое количество числовых типов**
- * Мы **умышленно упростили** и не стали выделять простые, составные и другие множества
- * При этом программирование **добавляет свои нюансы**



МАТЕМАТИКА VS ПРОГРАММИРОВАНИЕ

- * Программирование заставляет нас думать не только о «свойствах числа», но и **«как оно должно быть представлено в памяти»**
- * При этом еще играет важную роль **«есть или нет нативная поддержка процессором»**
- * Аппаратная поддержка операций над числовыми типами **значительно ускоряет работу с ними**

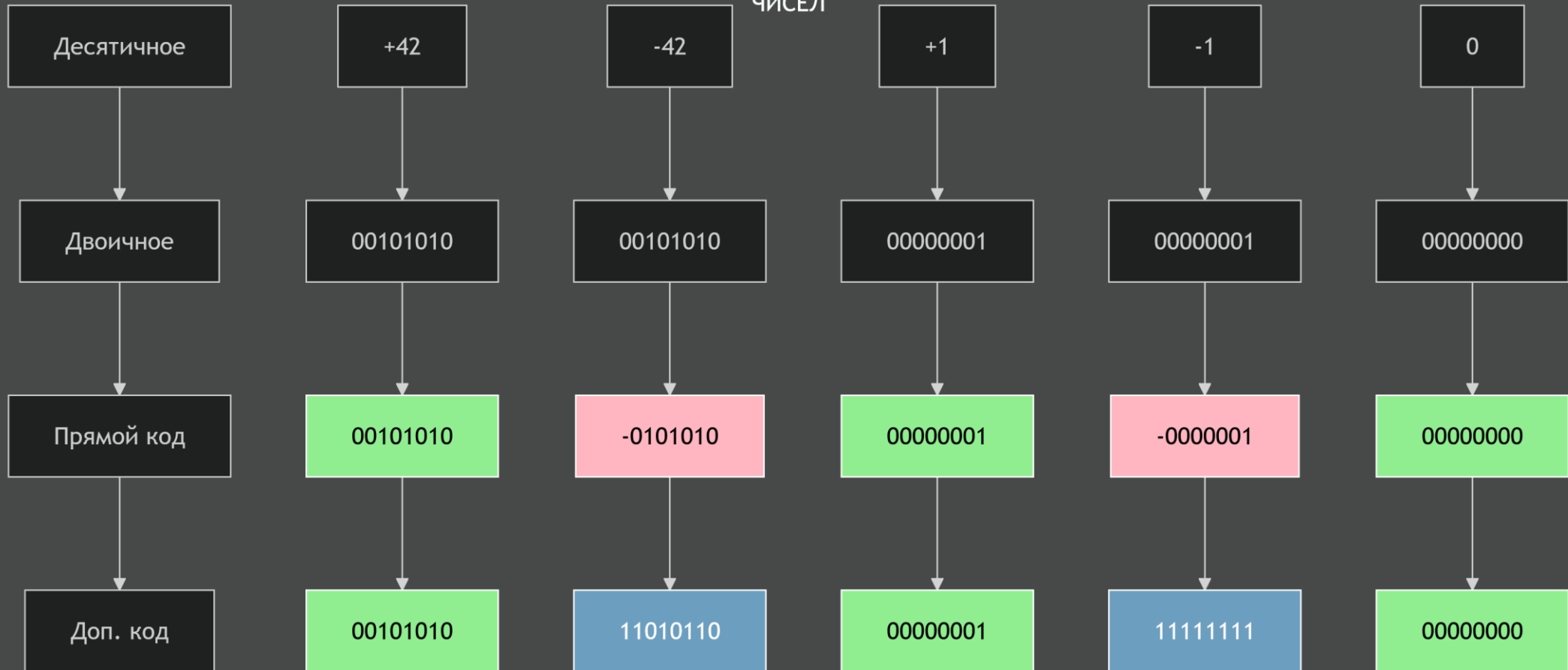
ЧТО ЕЩЕ ЗА
ДЕСЯТИЧНОЕ
ПРЕДСТАВЛЕНИЕ?



ВСПОМНИМ ПЕРВУЮ ЛЕКЦИЮ

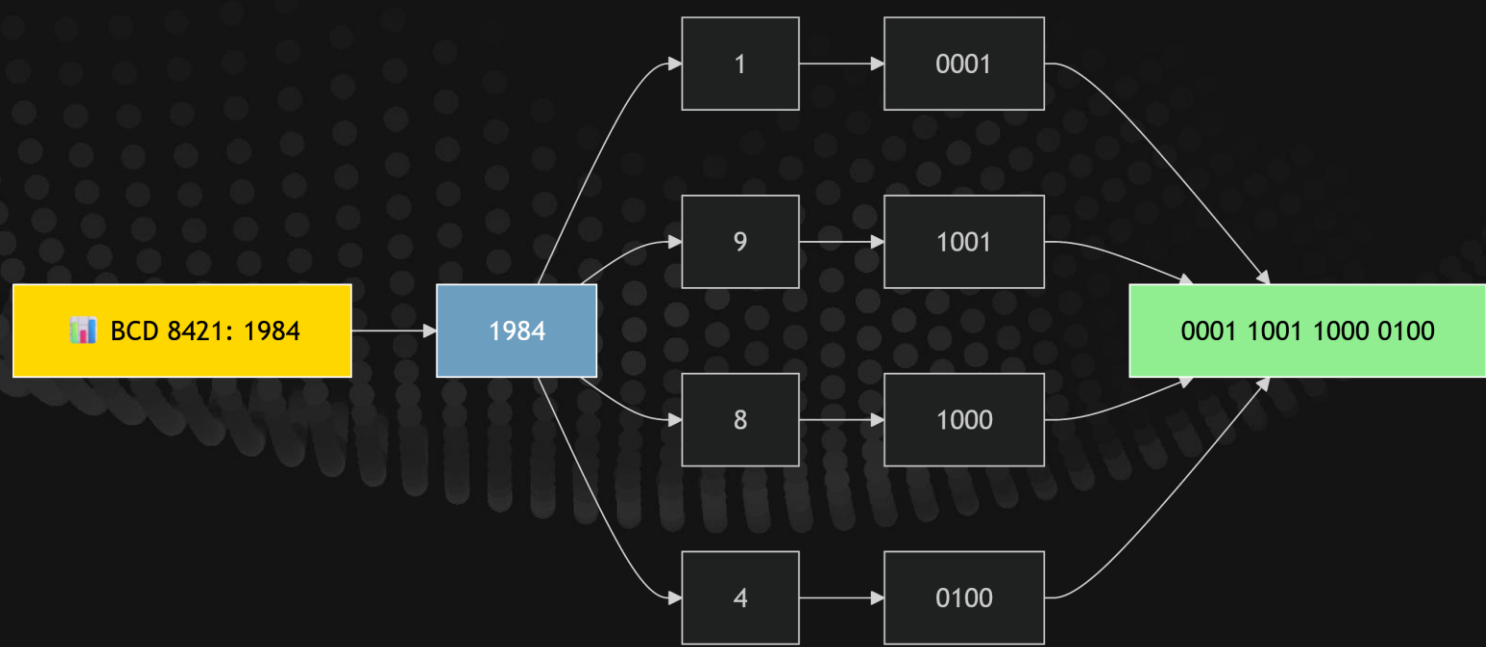
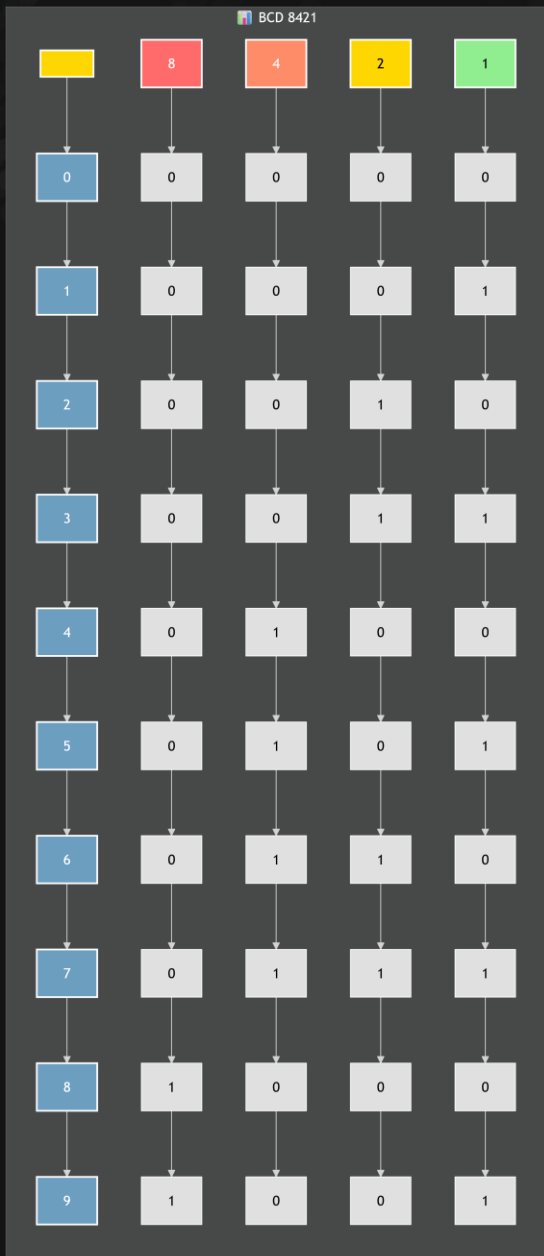
- * Мы научились представлять целые беззнаковые числа **в виде двоичного числа используя** алгоритм прямого кодирования
- * А знаковые целые используя дополнительный код (**тоже двоичное представление**)
- * Именно так в 90% случаев и представлены **целые числа** в большинстве языков программирования
- * Более того, процессоры поддерживают операции с такими числами **на аппаратном уровне**

ПРЕДСТАВЛЕНИЕ
ЧИСЕЛ



НО ЕСТЬ И ДРУГИЕ СПОСОБЫ КОДИРОВАНИЯ

- * Вместо того, чтобы представить десятичное число в виде двоичного, можно **закодировать десятичную форму в двоичный вид**
- * Например, дать каждой цифре от 0 до 9 свой код, а числа строить **как последовательности таких цифр**
- * Такой подход **напоминает представление строк в памяти** (последовательность символов)



РЕФЛЕКСИРУЕМ

- * Одна цифра в BCD 8421 занимает 4 бита и представляет собой **обычное кодирование прямым кодом**
- * При таком способе кодирования в одном байте **можно уместить значение не больше 99**
- * Что **значительно меньше подхода с прямым кодом** всего числа (256 против 99)
- * Но у такого подхода есть свои плюсы...

РАЗЛОЖЕНИЕ НА РАЗРЯДЫ

- * BCD число тривиальным образом раскладывается на разряды:
просто читаю по 4 бита цифру за цифрой
- * А вот с прямым кодированием все «сложнее» с точки зрения вычислений

```
function decompose(num) {  
  do {  
    console.log(num % 10); // Деление с остатком на 10 - дорогая операция  
    num = Math.floor(num / 10); // Деление на 10 - дорогая операция  
  } while (num > 0);  
}  
  
decompose(5421);
```

```
function decomposeBCD(num, len) {  
  for (let i = 0; i < len; i++) {  
    const mask = 0b1111 << i * 4; // Очень простая операция  
    console.log((num & mask) >> i * 4); // Очень простая операция  
  }  
}  
  
// Кодировем 5421 в BCD 8421  
const bcd5421 = 5 << 12 | 4 << 8 | 2 << 4 | 1;  
  
decomposeBCD(bcd5421, 4 /* Сколько цифр в нашем числе */);
```


И ЗАЧЕМ НАМ НУЖНО
РАССКЛАДЫВАТЬ
ЧИСЛО НА РАЗРЯДЫ?



ЭТО ВАЖНО ДЛЯ ОТОБРАЖЕНИЯ

- * Если мы рендерим число на экране или печатаем на бумаге, то в 90% мы хотим видеть его в 10-тичной системе счисления
- * Но в компьютере оно представлено в двоичном виде
- * Нам нужно на основе байт числа сформировать строку, которую уже затем отрендерить на экране или бумаге
- * Для формирования этой строки мы будем обходить число по разрядам
- * Для BCD-представления это делается элементарно
- * А для двоичных чисел – нет

ПОЧЕМУ ТОГДА
ИСПОЛЬЗУЮТСЯ
ДВОИЧНЫЕ
ЧИСЛА?



У НИХ БОЛЬШЕ ПЛЮСОВ

- * Лучше используется память, а процессору **проще делать арифметические операции**
- * Мы гораздо **больше делаем вычисления с числами**, чем выводим их на экран
- * Но если наша задача сводится к отображению чисел, а ресурсов мало, то **десятичное представление куда эффективнее**
- * Например, **программируем электронные часы**
- * В телефонии тоже используются **десятичные представления чисел** (похожий на BCD формат)

ВАЖНО ПОНИМАТЬ

- * У десятичных чисел своя арифметика, **отличная от двоичных**
- * В x86 процессорах **поддерживаются нативно** такие операции
- * Представление отрицательных чисел **тоже делается иначе**
- * Для BCD **используются алгоритмы дополнение до 9 или до 10**
- * Есть и **другие форматы** кодирования десятичных чисел кроме BCD

А В JS КАК ЭТО
ИСПОЛЬЗОВАТЬ?



РАЗБЕРЕМСЯ С ЧИСЛАМИ В JS

- * В JavaScript «на поверхности» у нас имеется два числовых представления: **number** и **bigint**
- * Формат **number** используется для всех «обычных» чисел и позволяет хранить знаковые дробные и целые числа размером 64 бита
- * Формат **bigint** используется для всех «больших» знаковых целых чисел без фиксированного размера
- * BCD числа или другие форматы десятичного кодирования нативно в JS не представлены

НАПРИМЕР



```
let a = 100500;  
console.log(typeof a); // number
```

```
let b = 100500.42;  
console.log(typeof b); // number
```

```
let c = 100500n;  
console.log(typeof b); // bigint
```

```
console.log(c + a);          // Исключение: Cannot mix BigInt and other types, use explicit conversions  
console.log(c + BigInt(a)); // 201000n
```

НО ВСД МОЖНО РЕАЛИЗОВАТЬ САМОСТОЯТЕЛЬНО

- * Можно кодировать байты в таком формате используя типизированные массивы
- * Или кодировать внутри `number` и `bigint`
- * Однако придется самостоятельно реализовать всю арифметику (если она нужна)

ОБРАТИТЕ ВНИМАНИЕ

- * Мы самостоятельно реализовывали `toString`, чтобы можно было **рендерить** такие числа на экране
- * По умолчанию там используется **формат двоичного кодирования**, поэтому при рендеринге наше число выглядит совсем иначе

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВЫДЫХАЕМ

- * Сейчас главное попытаться понять отличия в подходе кодирования таких чисел
- * И зачем они вообще нужны
- * Помимо простоты рендера у них есть еще преимущества, которые мы сегодня обсудим

ДРОБНЫЕ ЧИСЛА

- * Мы уже догадываемся, что дробные числа **кодируются иначе чем целые**
- * При этом там также есть деление на: **рациональные и иррациональные**
- * И на **двоичный и десятичный способ кодирования**
- * Тип **number** в JS используется способ кодирования любых вещественных чисел в виде двоичной дроби по стандарту [IEEE-754](#)
- * Такой способ **используется в большинстве современных ЯП**
- * **Нативно поддерживается процессором**

ФОРМАТ ПЛАВАЮЩЕЙ ТОЧКИ

- * Количество знаков после запятой **не регламентировано и может меняться** в рамках одного типа числа
- * Число в таком формате **представляется в специальной нормальной форме**
- * Например, $1234 == 1.234 * 10^{**3}$ или $12.34 == 1.234 * 10^{**1}$
- * Или $0.0012 == 1.2 * 10^{**-3}$
- * Где будет «стоять точка» **определяется степенью десятки**, поэтому точка и «плавающая»

ФОРМАТ ФИКСИРОВАННОЙ ТОЧКИ

- * При таком подходе **позиция точки строго регламентирована** в рамках одного типа числа
- * Максимальное **количество цифр ДО и ПОСЛЕ** запятой зафиксировано
- * Например, 2 ДО и 2 ПОСЛЕ подойдет для **12.34**
- * Из-за своей **статической природы** этот формат используется достаточно редко

КАК **number**
ВЫГЛЯДИТ
В ДВОИЧНОМ
ВИДЕ?



ПОХОЖЕ, НО НЕМНОГО ИНАЧЕ

- * Во-первых **нет смысла кодировать 1.** – это не может поменяться
- * Нужно закодировать «степень» или «порядок» (только **не 10-ки, а двойки**)
- * И **закодировать числа после .** – это называется «мантиссой»
- * И **не забыть про знак** – для этого просто выделяем крайне левый бит числа

 1.3 B DOUBLE



0

01111111111

010011001100110011001100



ЗНАК

1 бит

0 → положительное



ЭКСПОНЕНТА

11 бит

1023 → $2^0 = 1$



МАНТИСА

52 бита

1.3 - ϵ

```
function float64toString(num) {
  const float64 = new Float64Array([num]);
  const bytes = new Uint8Array(float64.buffer);

  let str = "";

  for (let i = bytes.length; i--;) {
    str += bytes[i].toString(2).padStart(8, "0");

    if (i !== 0) {
      str += "_";
    }
  }

  return str;
}
```

```
// 10111111_11110000_00000000_00000000_00000000_00000000_00000000_00000000
console.log(float64toString(-1));

// 01000010_01010100_00001110_01101101_01110011_11000000_00000000_00000000
console.log(float64toString(34.45656e10));

// 01111111_11110000_00000000_00000000_00000000_00000000_00000000_00000000
console.log(float64toString(Infinity));

// 11111111_11110000_00000000_00000000_00000000_00000000_00000000_00000000
console.log(float64toString(-Infinity));

// 01111111_11111000_00000000_00000000_00000000_00000000_00000000_00000000
console.log(float64toString(NaN));
```

РЕФЛЕКСИРУЕМ

- * Целиком **number** занимает 64 бита
- * Крайний левый бит хранит знак (1 означает минус)
- * Потом идут 11 бит **означающих порядок** (степень)
- * Порядок может быть положительным или отрицательным, чтобы получить реальное значение нужно **вычесть порядок из 1023**
- * Оставшиеся 52 бита отведены под мантиссу

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ПОНИМАЮ ВАШИ ЧУВСТВА

- * Данный способ кодирования является примером гениальности инженеров его придумавших
- * Нам важно понимать последствия такого способа кодирования

ТОЛЬКО 53 БИТА НА ЦЕЛЫЕ ЧИСЛА

- * Несмотря на размер мантиссы в 52 бита в `number` можно безопасно хранить целое до $2^{53} - 1$
- * Дальше начинаются переполнения и потеря точности
- * «Лишний» бит взялся, из-за того, что число находится в нормальной форме 1.



```
console.log(Number.MAX_SAFE_INTEGER === 2 ** 53 - 1);  
console.log(Number.MIN_SAFE_INTEGER === -(2 ** 53 - 1));  
console.log(2 ** 56 - 4 === 2 ** 56); // true из-за переполнения
```

ГРАНИЦА **number**

- * Граница числа находится от $\approx -1.7976931348623157 \times 10^{308}$ до $\approx 1.7976931348623157 \times 10^{308}$
- * Число **меньше этой границы** называет «отрицательной бесконечностью» [специальная константа]
- * А больше – «положительной бесконечностью» [специальная константа]
- * При этом важно понимать, что мантисса физически не может хранить число больше **$2^{53} - 1$**
- * Значит в какой-то момент **начнутся проблемы с точностью**

МОЖНО ЛИ
ДЕЛИТЬ ТАКИЕ
ЧИСЛА НА 0?



ДА

- ✱ В отличие от целых чисел, стандарт [IEEE-754](#) явно разрешает такие операции
- ✱ А при деления 0 на 0 результатом будет *NaN* (специальная константа)

```
console.log(42 / 0 === Infinity);  
console.log(-42 / 0 === -Infinity);  
  
console.log(NaN !== NaN); // NaN не равен сам себе  
console.log(Number.isNaN(0 / 0));
```

ЕСТЬ ПОЛЕЗНЫЙ API



```
console.log(Number.isFinite(Infinity)); // false  
console.log(Number.isInteger(42.2)); // false  
console.log(Number.isSafeInteger(2e56)); // false
```

```
console.log(isNaN(NaN)); // true  
console.log(isNaN("foo")); // true  
console.log(isNaN("42")); // false
```

```
console.log(Number.isNaN(NaN)); // true  
console.log(Number.isNaN("foo")); // false  
console.log(Number.isNaN("42")); // false
```

-0 и +0

- ✱ Отрицательные числа в [IEEE-754](#) кодируются **отличным от целых чисел образом**
- ✱ У этого есть артефакт в виде наличия «двух нулей»
(`console.log(Object.is(0, -0)); // false`)

ПОТЕРЯ ТОЧНОСТИ ИЗ-ЗА ДВОИЧНОЙ СПЕЦИФИКИ

- * Люди привыкли работать с десятичными дробями, но в компьютере такие дроби работают не очень эффективно
- * Поэтому в формате [IEEE-754](#) используются степени двойки, а не десятки
- * У этого есть последствия



```
console.log(0.1 + 0.2 === 0.3); // false, и JS тут не причем  
console.log(0.1 + 0.2 === 0.30000000000000004); // true
```

12 СТЕПЕНИ ДВОЙКИ

$$2^{10} = 1024$$

$$2^{20} = 1,048,576$$

$$2^{30} = 1,073,741,824$$

$$2^{40} = 1,099,511,627,776$$

РАСХОЖДЕНИЕ

+2.4%

+4.86%

+7.37%

+9.95%

10 СТЕПЕНИ ДЕСЯТКИ

$$10^3 = 1000$$

$$10^6 = 1,000,000$$

$$10^9 = 1,000,000,000$$

$$10^{12} = 1,000,000,000,000$$

ЧЕМ МЕНЬШЕ РАЗМЕР МАНТИСЫ – ТЕМ СИЛЬНЕЕ ПОГРЕШНОСТЬ

- * Тип `number` в JS имеет размер 64 бита, где мантисса занимает 52 бита
- * Такой формат называют число с плавающей точкой **двойной точности** (Double)
- * Но число с плавающей точкой можно хранить **и в 32 или, даже, в 16 битах**
- * Однако, это увеличивает погрешность таких чисел
- * Для особо критичных вещей используют даже **числа с размером 128 бит**

```
console.log(new Float16Array([1.3])[0] === 1.2998046875);  
console.log(new Float32Array([1.3])[0] === 1.2999999523162842);
```

ЕСТЬ ЗАДАЧИ, ГДЕ ТОЧНОСТЬ ДОЛЖНА БЫТЬ 100%

- * Например в финтехе **даже малейшее изменение** в числе могут иметь последствия
- * Для решения этой проблемы **используются десятичные дроби**
- * Например, можно использовать **уже знакомые BCD и фиксированную точку**
- * На практике так зачастую и делают

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

И СНОВА Я ПОНИМАЮ ВАШИ ЧУВСТВА

- * Но мы не можем игнорировать особенности кодирования дробных чисел
- * Ошибки такого рода очень сложно обнаружить, а последствия могут быть фатальны
- * Я бы сказал, что на этом нюансы с **number** закончились, но это не так

В JS ДИНАМИЧЕСКА ТИПИЗАЦИЯ

- * Информация о любых типах тащится в рантайм, а до этого **«все типы условно равны»**
- * Это **не просто высказывание**, а требование архитектуры процессора – нам нужно знать сколько байт в памяти занимает каждый тип
- * При динамической типизации все существующие значения представляются **указателями** на область памяти, где лежат данные
- * **Размер указателя статичен** и обычно равно 32 или 64 бита
- * Все в JS – это указатели

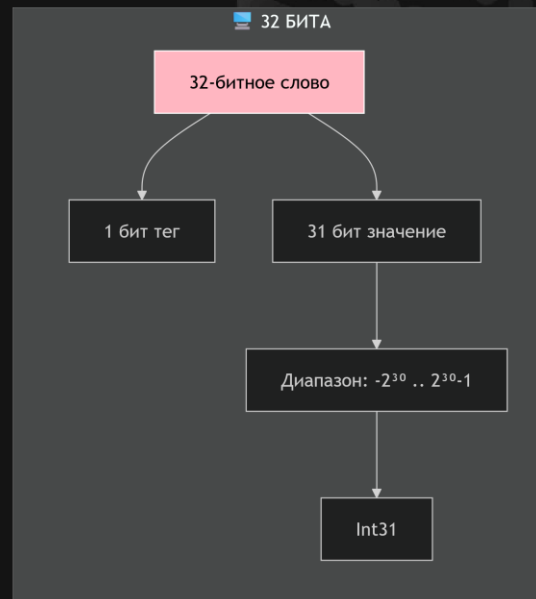
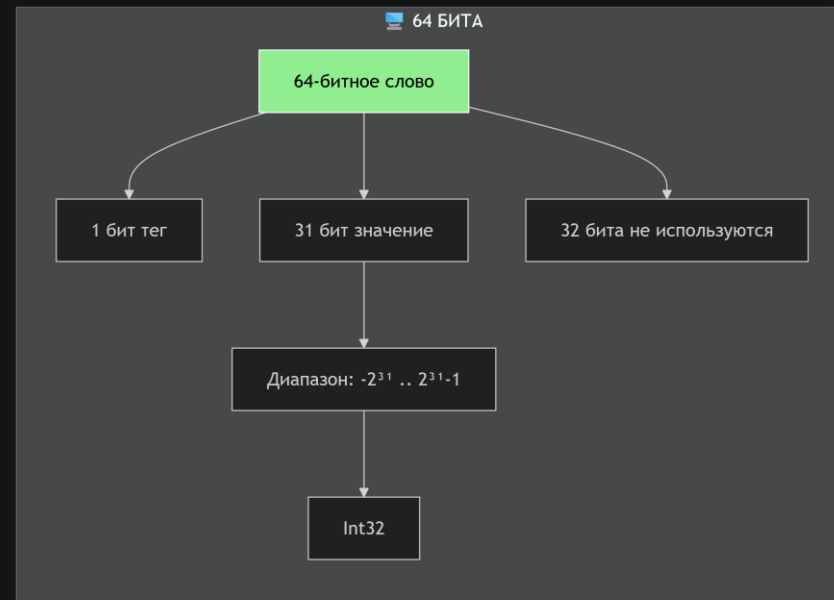
НО ЭТО ПОЛОВИНА ПРАВДЫ

- * Хранить значения типов `true false null undefined` по указателю было бы чудовищно не эффективно
- * Поэтому они реализуются как «константные указатели», которые, фактически, никуда не ссылаются
- * Получается, что сам указатель на данные таких типов – и есть сами данные
- * Целые знаковые числа типа `Int32/Int31` тоже кодируются так
- * Такой формат называется SMI (Small Integers)

node --allow_natives_syntax



```
console.log(%DebugPrint(-42));      // DebugPrint: Smi: 0xffffffffd6 (-42)
console.log(%DebugPrint(2 ** 31 - 1)); // DebugPrint: Smi: 0x7fffffff (2147483647)
console.log(%DebugPrint(2 ** 31));  // DebugPrint: 000000618AC0D429: [HeapNumber]
console.log(%DebugPrint(42.10));    // DebugPrint: 0000013A30D8D439: [HeapNumber]
```

НО ВЕДЬ В
СПЕЦИФИКАЦИИ
ТИП *number*
СТРОГО
РЕГЛАМЕНТИРОВАН?

ДА

- ✱ Представление через SMI в спецификации не значит
- ✱ Это конкретная **оптимизация** на уровне виртуальной машины
- ✱ На практике такое представление используется всеми (спасибо Smalltalk)

ЦЕЛЫЕ
number
ЭФФЕКТИВНЕЕ?



ДА, НО ТОЛЬКО ЕСЛИ ОНИ МЕНЬШЕ 2 ** 31

- * Они потребляют меньше памяти (меньше само число + не нужен лишний указатель)
- * Могут размещаться на стеке и передаются по значению, а не ссылке
- * Лучше используют кэш процессора


```

const SIZE = 10_000_000;

const smi = new Array(SIZE);
const nonSmiFloat = new Array(SIZE);
const nonSmiInt = new Array(SIZE);

for (let i = 0; i < SIZE; i++) {
    smi[i] = i % (2 ** 31 - 1); // SMI
    nonSmiFloat[i] = (i % 1000) + 0.5; // Дробные (HeapNumber)
    nonSmiInt[i] = i + 2_500_000_000; // > 231 (HeapNumber)
}

const createSum = () => (arr) => {
    let s = 0;
    for (let i = 0; i < arr.length; i++) { s += arr[i]; }
    return s;
};

// Разные функции, чтобы не путать JIT
const sumSmi = createSum(); const sum = createSum();

// Прогрев
for (let i = 0; i < 10; i++) { sumSmi(smi); sum(nonSmiFloat); sum(nonSmiInt); }

let start = performance.now();
sumSmi(smi);
console.log(`SMI (целые) : ${performance.now() - start}.toFixed(2)} мс`);

start = performance.now();
sum(nonSmiFloat);
console.log(`He SMI (дробные) : ${performance.now() - start}.toFixed(2)} мс`);

start = performance.now();
sum(nonSmiInt);
console.log(`He SMI (целые) : ${performance.now() - start}.toFixed(2)} мс`);

```

```

node --jitless smi-bench.js
# SMI (целые) : 96.31 мс
# He SMI (дробные) : 103.44 мс
# He SMI (целые) : 103.32 мс

```

```

node smi-bench.js
# SMI (целые) : 3.83 мс
# He SMI (дробные) : 4.22 мс
# He SMI (целые) : 3.89 мс

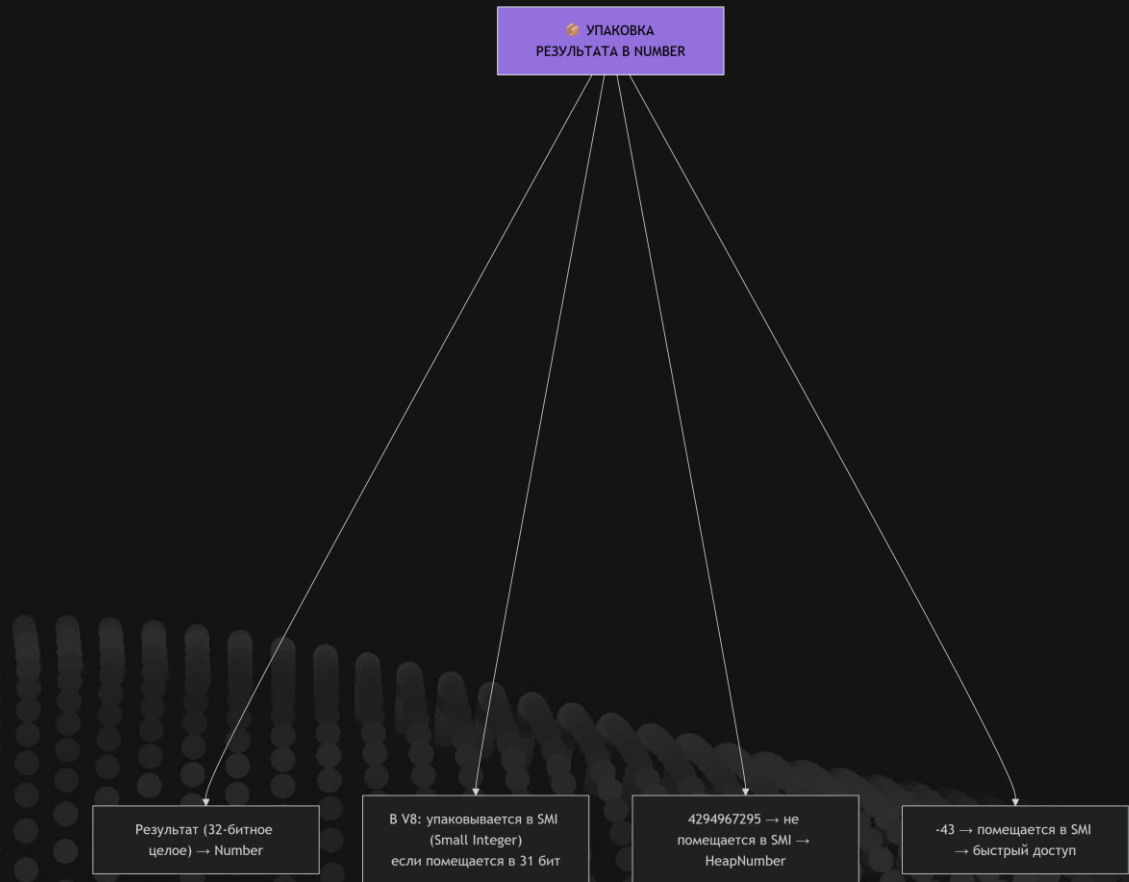
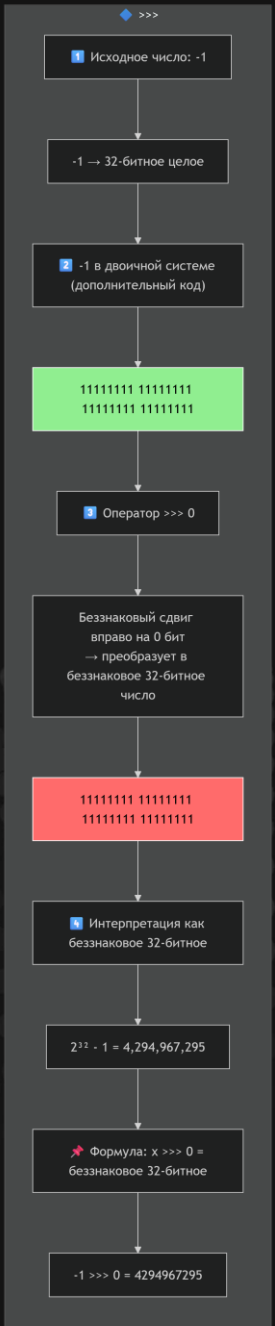
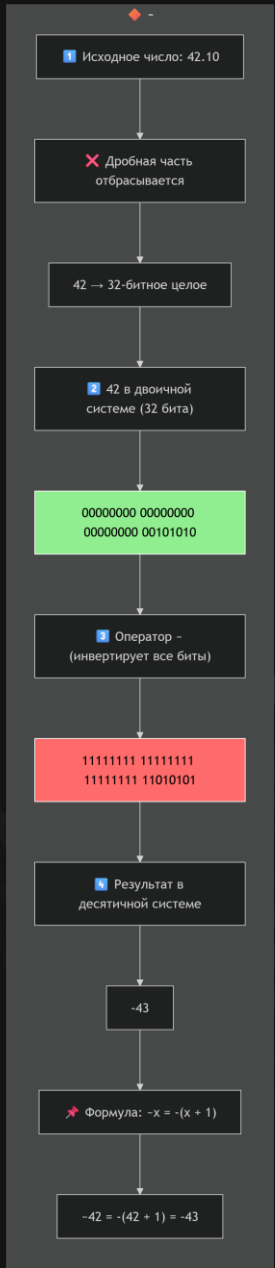
```

ВЫВОДЫ

- * Даже с учетом JIT SMI числа **работают немного быстрее**
- * Но не спешите переписывать все на SMI числа – есть **более правильные способы для этого**
- * Нам важно **понимать природу такого эффекта**
- * К сожалению, это не все нюансы с **number**

number И ПОБИТОВЫЕ ОПЕРАТОРЫ

- * В JS нам доступно множество побитовых операторов: `& | << >> >>> ~ ^`
- * Работают они **только с целыми числами**
- * Когда мы применяем такой оператор к `number`,
то его представление преобразовывается в `Int32/Uint32`
- * Затем вычисляется операция, а **результат** обратно упаковывается в `number`



ПОЧЕМУ ЭТО ВАЖНО

- * Побитовые операторы переносят нас из мира чисел с плавающей точкой **в мир целых чисел**
- * Такие числа используют прямой и дополнительный коды
- * Например, `console.log(42 === ~-42 + 1)`
- * Или `console.log(1 << 2 === 4)` // `0001 << 2 == 0100 == 4`
- * **Зачем это нужно** поговорим в четверг 😊

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЖКА, СТОЖКААА!

ВЫДЫХАЕМ

- * Мы разобрали все важные для нас моменты с `number`
- * Теперь можно перейти к `bigint`

БОЛЬШИЕ ЗНАКОВЫЕ ЦЕЛЫЕ ЧИСЛА

- * Главная особенность `bigint` в том, что занимаемая типом память **может меняться динамически от конкретного значения**
- * Это позволяет хранить очень большие числа **без страха числового переполнения**
- * Фактически, значение в `bigint` ограничено лишь **доступной памятью виртуальной машины**
- * Ценою такой гибкости является эффективность – `bigint` намного медленней `number`

НЮАНСЫ **bigint**

- * Нельзя применять арифметические операции между **bigint** и **number** без явного преобразования типов
- * Например, `console.log(2n + BigInt(10.30))`
- * При преобразовании типов может быть отброшена дробная часть или случиться переполнение
- * Побитовые операторы с **bigint** не делают преобразование к `Int32/Uint32`

```

const SIZE = 10_000_000;

const numberArray = new Array(SIZE);
const bigintArray = new Array(SIZE);

for (let i = 0; i < SIZE; i++) {
  numberArray[i] = i;
  bigintArray[i] = BigInt(i);
}

function numberSum(arr) {
  let s = 0;
  for (let i = 0; i < arr.length; i++) { s += arr[i]; }
  return s;
}

function bigintSum(arr) {
  let s = 0n;
  for (let i = 0; i < arr.length; i++) { s += arr[i]; }
  return s;
}

// Прогрев
for (let i = 0; i < 10; i++) { numberSum(numberArray); bigintSum(bigintArray); }

let start = performance.now();
numberSum(numberArray);
console.log(`number : ${performance.now() - start}.toFixed(2)} mc`);

start = performance.now();
bigintSum(bigintArray);
console.log(`bigint : ${performance.now() - start}.toFixed(2)} mc`);

```

```

node --jitless bigint-bench.js
# number : 99.33 mc
# bigint : 105.97 mc

```

```

node bigint-bench.js
# number : 5.37 mc
# bigint : 26.91 mc

```

ВЫВОДЫ

- * С использованием JIT **bigint** значительно медленней **number**
- * Чем больше число в **bigint** – тем медленней оно будет работать
- * Это инструмент для конкретных задач, например, для блокчейна

ПОЧТИ ВСЕ

- ✱ Мы разобрали все числовые типы в JS, но **есть еще кое-что**
- ✱ В JS есть множество типизированных массивов, где **хранемое внутри значение следует правилам кодирования конкретных типов**

ОГЛАСИТЕ ВСЕ ССПИСОК

💧 ВЕЩЕСТВЕННЫЕ

Float16Array
2 байта 🏷️

Float32Array
4 байта

Float64Array
8 байт

💎 ЦЕЛОЧИСЛЕННЫЕ

Int8Array
1 байт

Uint8Array
1 байт

Uint8ClampedArray
1 байт

Int16Array
2 байта

Uint16Array
2 байта

Int32Array
4 байта

Uint32Array
4 байта

BigInt64Array
8 байт

BigUint64Array
8 байт



```
const arr1 = new Float32Array([1, 2, 3, 4, 5]);
const arr2 = new Int32Array([1, 2, 3, 4, 5]);

// ArrayBuffer {
//   [Uint8Contents]: <00 00 80 3f 00 00 00 40 00 00 40 40 00 00 80 40 00 00 a0 40>,
//   [byteLength]: 20
// }
console.log(arr1.buffer)

// ArrayBuffer {
//   [Uint8Contents]: <01 00 00 00 02 00 00 00 03 00 00 00 04 00 00 00 05 00 00 00>,
//   [byteLength]: 20
// }
console.log(arr2.buffer)
```

ТУТ ОЧЕНЬ ВАЖНО ПОНИМАТЬ

- * «Снаружи» у нас есть только **number** и **bigint** – значит любое число «читаемое» из такого массива **будет преобразовано в один из двух типов**
- * При записи в такой массив числа – оно **будет конвертироваться во внутреннее представление**

НАПРИМЕР

```


// 1. Знак интерпретируется как обычный бит
// 2. Дробная часть отбрасывается
// 3. Переполнение
const arr = new Uint8Array([-10, 42.19, 321]);

// 246, 42, 65
for (const el of arr) {
  console.log(el);
}
```

В ЧЕМ СМЫСЛ

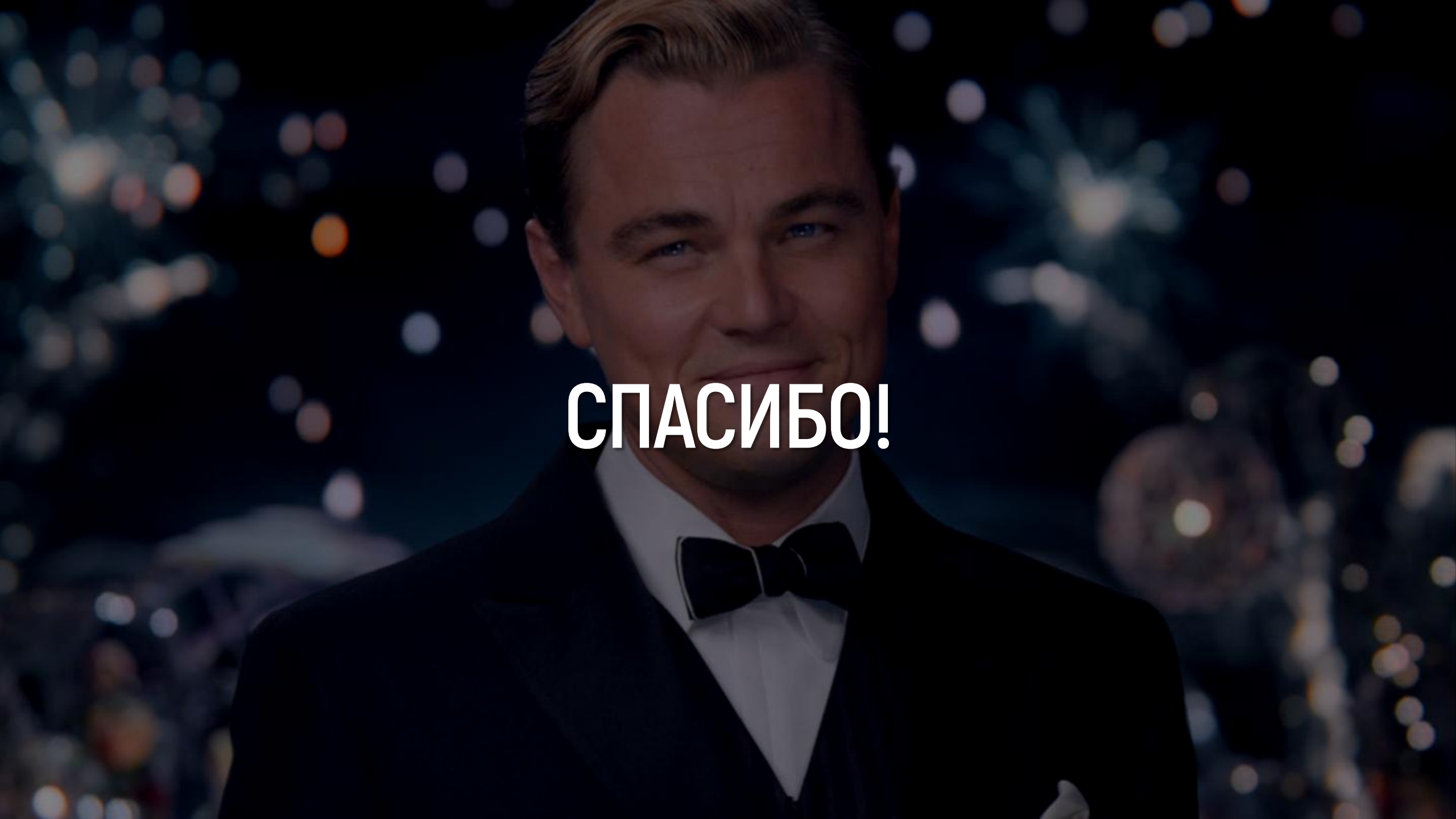
- * Типизированные массивы позволяют **уйти от ограничений динамической типизации**
- * Внутри хранятся не указатели, а **конкретные значения**
- * Контролируем выделяемую память, кэш процессора и работу сборщика мусора
- * Мы разберем все нюансы и плюсы **на отдельной лекции по ним**

```
// Массив указателей на Double
const a = [42.10, 5.12];

// Массив значений вида Double
const b = new Float64Array([42.10, 5.12]);
```


ПОДВЕДЕМ ИТОГИ

- * В математике и программировании **существуют множество различных видов чисел**
- * Но и **одно и тоже число** можно по разному представить в памяти
- * В JS у нас есть два строенных числовых типа: **number** и **bigint**
- * Тип **number** реализуется как **число с плавающей точкой двойной точности**
- * **Есть нюансы** с хранением небольших целых чисел и работе с побитовыми операторами
- * Тип **bigint** позволяет хранить **очень большие целые числа**
- * А еще есть типизированные массивы

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with out-of-focus bokeh lights in various colors (white, yellow, orange, blue), suggesting a night-time event or fireworks.

СПАСИБО!